SCOTTISHPOWER

Elastic Stack Cloud Operations Runbook

Monitoring • Maintenance • Incident Response • Recovery

Elastic Cloud Hosted | AWS eu-west-2 | Production and other approved environments

Document control	Value
Document ID	SP-OBS-ELASTIC-RUNBOOK (proposed; confirm)
Version/status	0.9 - review draft; not approved for production use
Date	20 October 2026
Business owner	[CONFIRM]
Technical owner	[CONFIRM]
Approver / review cycle	[CONFIRM] / quarterly and after major incidents or platform changes
Classification	Internal - review against ScottishPower information handling policy
Related documents	Service inventory; escalation matrix; incident management; change management; resilience policy; access standards

CONTROL POINT This is a draft based on screenshots of an existing runbook and team review comments. Service URLs, node counts, RTO/RPO, alert thresholds, supported versions, ownership and approved escalation routes must be verified by ScottishPower before operational sign-off.

Who should use this runbook

Service desk and non-technical coordinators use the plain-language symptoms, decision rules, ticket guidance and escalation steps. Observability engineers and authorised administrators use Kibana, the Elastic Cloud console and the read-only diagnostic APIs. Write operations require an approved change or incident authorisation.

Contents

- 1. How to use this runbook and scope
- 2. Elastic Stack explained in everyday language
- 3. ScottishPower cloud architecture and service inventory
- 4. Operating model, security boundaries and escalation
- 5. Operating cadence and health-check matrix
- 6. Daily operations: detailed checks (D01-D12)
- 7. Weekly, monthly and quarterly maintenance
- 8. Incident and troubleshooting playbooks (IR01-IR12)
- 9. Planned change: upgrades, onboarding and decommissioning
- 10. Backup, restoration and disaster recovery
- 11. Thresholds, alerts, evidence and reporting
- 12. Acceptance checklist, open items and glossary
- Appendix A. Read-only diagnostic command catalogue
- Appendix B. Reference standards and official documentation

WORKING CONVENTION Text in [SQUARE BRACKETS] indicates local ScottishPower details to supply or verify. Procedures use the current Elastic Cloud Hosted UI where possible; individual menu names vary by stack version and permissions.

1. How to use this runbook and scope

1.1 Purpose

Provide a consistent, auditable method to operate the ScottishPower Elastic Cloud Hosted observability service. The procedures aim to detect and resolve issues early, reduce missing monitoring data, protect service availability and support controlled recovery.

1.2 Scope

- Elastic Cloud Hosted deployment health, node roles, Kibana and Integrations Server/Fleet Server (where present).
- Data ingestion from Elastic Agent, Filebeat, Metricbeat, Auditbeat, Logstash and approved integration sources.
- Logs, metrics, APM traces, synthetic availability checks, dashboards, alerts and data quality.
- Elasticsearch indices/data streams, lifecycle management, shard allocation, hot/warm data tiers and storage growth.
- Snapshots, recovery assurance, platform and component upgrades, security and access reviews.
- New source onboarding, changes, retirement and operational handover.

1.3 Out of scope

Underlying cloud-host hardware management, AWS infrastructure administration performed by Elastic, application source-code fixes and organisation-wide identity or networking changes. These are handled through the appropriate service provider or owning team. The runbook must be linked to those external procedures.

1.4 Ground rules

- Begin in READ-ONLY mode. Never run DELETE, PUT, POST, _reroute, _close, _open, _rollover, _ilm/retry, _snapshot restore or policy edits without the required authorisation.
- First determine whether the issue is a platform failure, a collection/ingest failure or an application failure. A green cluster does not prove that log data is arriving.
- Always scope checks to the correct deployment, Kibana Space, environment, index/data stream and time range.
- Capture evidence BEFORE changing a configuration: time, errors, metrics, affected systems and recent changes.
- Use organisation-approved SSO and least-privilege access. Never place API keys, session cookies or passwords into tickets or the runbook.
- If an Elastic Cloud deployment is down or a managed component has a provider-side fault, open an Elastic support case through the agreed support process and connect it to the internal incident.

1.5 Quick triage for non-technical responders

What the user reports	First place to check	What it may mean	Next action
Cannot access Kibana	Kibana sign-in URL and deployment status	Service outage, SSO or permissions fault	Check Cloud deployment events, scope of users, raise incident
Dashboards open but no new logs	Discover → correct data view & time	Source, agent, pipeline or timestamp fault	Follow IR04 missing-data playbook
Monitoring alerts or pages slow	Stack Monitoring and Cloud performance	Capacity, slow queries, workload peak	Follow IR02/IR03
Agents shown offline	Fleet → Agents	Host/service/network	Follow IR05

What the user reports	First place to check	What it may mean	Next action
		unavailable	
Application not visible in APM	Observability → APM/Services	Instrumentation or ingest issue	Follow IR08
Red/yellow cluster health	Stack Monitoring or cluster health API	Lost replicas or primary shard problem	Follow IR01; escalate red promptly

2. Elastic Stack explained in everyday language

Elastic Stack collects operational signals from systems, stores and searches them, and turns them into dashboards and alerts. Think of it as a central monitoring library: collectors deliver records; processing validates and labels them; Elasticsearch stores/searches them; Kibana is the reading room; alerts tell teams when something goes wrong.

Term	Plain-English explanation	Example
Log	A written event about something that happened	Application login failed at 10:32
Metric	A numeric measurement over time	CPU 68%, memory 6 GB
Trace / APM	The journey of a request through an application	Payment request spends 2s on database call
Elasticsearch	The search-and-storage engine	Find all login errors in last hour
Kibana	The web interface for dashboards, search and management	Open a service availability dashboard
Elastic Agent / Beats	Software collecting telemetry from hosts and services	Collect Windows events or Linux logs
Fleet / Fleet Server	Central policy and status management for Elastic Agents	Identify 12 offline agents
Logstash	Optional pipeline that transforms/routes events	Convert source field to a standard name
Ingest pipeline	Elasticsearch-side processing before indexing	Parse raw message into structured fields
Index / data stream	Data organised for searching and retention	logs-system.syslog-prod
Shard / replica	Pieces of an index and redundant copies	Replica enables recovery when a node fails
ILM / data lifecycle	Rules to move, retain and delete older data	Keep hot 7 days, then warm 30 days
Snapshot	A recoverable point-in-time backup of eligible cluster data	Restore after an accidental deletion
Synthetics	Scheduled checks that act like an external user	Check URL returns HTTP 200
Alert rule	A test that creates a warning when conditions are met	Missing data for 15 minutes

2.1 Data flow: normal operating path

1. Produce	2. Collect	3. Process	4. Store	5. Use
Hosts, apps, network and cloud services	Elastic Agent, Beats, APM SDK, Logstash or API	Integrations, Logstash or ingest pipelines	Elasticsearch hot/warm data streams and indices	Kibana dashboards, APM, Synthetics, alerts

A failure at any stage may cause missing data. For example, the cluster can be healthy while a particular Filebeat collector is stopped. Hence health checks must cover both the platform and end-to-end data freshness.

3. ScottishPower cloud architecture and service inventory

3.1 Hosting model and observed design

The supplied runbook describes an Elastic Cloud Hosted deployment on AWS in the eu-west-2 region, using three availability zones (eu-west-2a, eu-west-2b and eu-west-2c). Its diagram shows hot-data, warm-data, master, ML, Integrations Server and Kibana instance roles. The diagram is a logical snapshot from the existing document, not a validated deployment plan.

Zone in screenshot	Roles depicted	Required verification
eu-west-2a	Hot data; warm data; Integrations Server; Kibana	Check current instance count and sizes in Cloud
eu-west-2b	Warm data; master-eligible	Confirm master quorum and zone resilience
eu-west-2c	Hot data; ML	Confirm actual ML capacity and HA requirements

ARCHITECTURE ASSURANCE A single master box in an illustrative diagram must not be interpreted as a sufficient highly available master configuration. Confirm the actual master-eligible/voting topology, zone distribution, node sizes, data-tier placement and replica policy in Elastic Cloud. Zone and role layout may change after scaling or upgrades.

3.2 Component responsibility

Component	What it does	Operational accountability
Elasticsearch hot tier	Handles newly written/search-intensive data	Cloud health, write performance, disk, shard balance
Elasticsearch warm tier	Keeps less frequently accessed history	Tier allocation, lifecycle, storage
Master-eligible nodes	Maintain cluster coordination and metadata	Quorum, stability, cluster state
Kibana	Search, dashboards, alerts and management UI	Login, saved objects, performance, access
Integrations/Fleet Server	Manages Elastic Agent policy communication; endpoint may host ingest as configured	Fleet connectivity, policy rollout, capacity
Machine learning	Anomaly detection or advanced analytics if enabled	Job states, capacity and licensing

Component	What it does	Operational accountability
Elastic Agent / Beats	Collect host telemetry	ScottishPower/source owners manage installation and configuration
Logstash	Optional customer-operated ETL processing	Hosting team monitors processes, pipelines, queue and outputs
APM agents and Synthetics	Application tracing and automated availability checks	Application owners and observability teams

3.3 Validated service register (complete before approval)

Environment	Cloud deployment ID / console link	Kibana / Elasticsearch / Fleet / APM	Owner and criticality
Production	[CONFIRM]	[CONFIRM 4 service URLs from live deployment]	[CONFIRM]
Non-production	[CONFIRM]	[CONFIRM]	[CONFIRM]
Monitoring deployment	[CONFIRM if separate]	[CONFIRM if used]	[CONFIRM]

The screenshots also reference Elastic Cloud Console, Elastic Support, documentation and community forums. Internal service URLs have intentionally not been transcribed from the photographed document; copy and validate them from the live deployment and approved CMDB rather than relying on a screenshot.

3.4 Operational inventory fields

For every host, agent, application and integration, record: application/service name; hostname or asset ID; environment; business unit; service owner; support group; data type; source technology; collection method; Fleet policy or Beats config; dataset/data stream; expected frequency; monitoring criticality; retention policy; PII classification; last-seen time; onboarding date; decommission status; and CMDB/change reference.

4. Operating model, security boundaries and escalation

4.1 Responsibility matrix (proposed; agree locally)

Activity	ScottishPower service/operations	Elastic support	Application or infrastructure owner
Check dashboards and raise incidents	Accountable and performs	Consult if hosted-side fault	Informed / consulted
Scale/upgrade hosted deployment	Request, authorise and validate	Runs/assists managed operations	Validate downstream compatibility
Agent, Beats and Logstash maintenance	Coordinate and verify	Consult for vendor defects	Own host changes or managed pipeline as agreed
Data quality and tag standards	Define and verify	Consult for stack issues	Own source data corrections
Access and API-key reviews	Own approvals and review	Platform support only	Approve service need
Restore and disaster recovery	Own business decision and test	Assist with ECH platform restore	Validate application data and functionality
Incident communications	Own internal timeline and	Provide case updates	Support diagnosis and fixes

Activity	ScottishPower service/operations	Elastic support	Application or infrastructure owner
	stakeholders		

4.2 Security and role boundaries

- Elastic Cloud ORGANISATION access to the Cloud console is distinct from deployment-level Kibana/Elasticsearch role privileges. Review both scopes.
- Use SSO/SAML where approved, MFA via corporate identity policy, periodic joiner/mover/leaver reviews, individual identities and tightly controlled break-glass access.
- Elasticsearch API keys for data-plane operations are not interchangeable with Elastic Cloud organisation API keys for control-plane APIs. Keep separate credential ownership and rotation schedules.
- Runbooks should reference secret vault paths and named owners, never embed secrets. Redact hostnames, email addresses, document payloads and personal data in supplier cases unless approved.
- Prefer narrowly scoped read-only roles for checks. Create/rotate/revoke API keys only through approved access requests and log evidence of the change.

4.3 Escalation and severity

Priority (example)	Trigger	Initial response route	Communication
P1 / Critical	Production platform unavailable, red primary-shard failures affecting critical data, widespread data loss	Major incident process + Elastic critical case	As per organisation P1 SLA
P2 / High	Sustained partial outage, critical ingest stalled, data at imminent write-block risk	On-call operations, source owner, vendor when required	As per P2 SLA
P3 / Medium	Single non-critical host or integration impaired; no broad business impact	Service owner work queue	Routine progress
P4 / Low	Cosmetic, documentation, improvement or planned review	Change/problem backlog	Normal governance

APPROVAL NEEDED Priority definitions, support channels, on-call contacts, SLA/OLA timings, RTO, RPO and escalation timeouts must come from ScottishPower policy; the example above must not override a published incident process.

5. Operating cadence and health-check matrix

Organise checks by frequency so the team can quickly distinguish routine prevention from deep problem investigation.

Cadence	Checks	Expected output
Daily / shift start	D01-D12: Cloud/cluster, nodes, shards, heap/CPU, storage, indexing, snapshots, ILM, source freshness, Fleet/Beats,	Daily health record; incident for red/failed checks

Cadence	Checks	Expected output
	APM/Synthetics, alerts	
Weekly	Agent version and connectivity trends; pipeline failures; tag completeness; index/retention anomalies; alert quality; capacity trend	Action list and owners
Monthly	Security/access/API key review; growth forecast; backup/restore evidence; integration version review; patch risk assessment	Monthly operational report and change backlog
Quarterly	Supported-version roadmap; resilience tabletop/restore test; policy and dependency review; ownership inventory; escalation drill	Quarterly service assurance sign-off
Event-driven	Upgrade, new integration, source decommission, outage, failed snapshot, large change, detected security event	Approved change record or incident/postmortem

5.1 Standard check recording format

Log Date/time (UTC), environment, operator, control ID, observed value, expected range, status (Green/Amber/Red/N/A), affected service(s), supporting link, ticket ID, corrective action owner, next review time and closure approval. Explain every N/A entry; do not mark a failed metric green only because the main cluster is green.

6. Daily operations: detailed checks (D01-D12)

For each procedure: where to go, simple meaning, what success looks like and what to do when the check fails. Authorized engineers may supplement these UI checks with read-only commands from Appendix A.

D01 Elastic Cloud deployment availability

Why it matters: The hosted environment must be running before dashboards or applications can use it.

Navigation: Elastic Cloud Console → Hosted deployments → [ScottishPower deployment] → Overview / Activity

Verification steps:

1. Select the PRODUCTION deployment and verify its region, version and deployment identifier.
2. Review deployment health, running instances, recent plan changes and any failed activity.
3. Open the registered Kibana URL and confirm successful loading and approved SSO access.
4. Check whether an ongoing planned maintenance/change explains any temporary status.

Expected outcome: Deployment is available, expected components are running and no unexplained failed activity is present.

If not OK: If deployment is unavailable, log timestamps and error details; check provider events; raise internal incident and Elastic support case as required.

Owner: Cloud Operations | Evidence: Deployment screenshot, changes/events and incident reference

D02 Cluster health and node availability

Why it matters: Green means primary and replica shards are allocated; red can mean some primary data is unavailable.

Navigation: Kibana → Stack Monitoring → Elasticsearch → Overview / Nodes; Dev Tools → GET /_cluster/health

Verification steps:

1. Verify cluster status, number of data nodes, node count and unassigned shards.
2. Compare current roles and counts against the approved architecture inventory.
3. Check for repeated node restarts, CPU pressure, JVM pressure or abnormal activity.

Expected outcome: Green in normal production operation; expected node roles and capacity present; no unexpected unassigned shards.

If not OK: Yellow: identify replica allocation issues and resilience impact. Red: escalate immediately; follow IR01. A green cluster is not proof of healthy data ingestion.

Owner: Observability Operations | Evidence: Cluster status, node count, recent event history

D03 Shard allocation and master stability

Why it matters: Shards are pieces of stored data. Unassigned replicas reduce resilience; unassigned primaries can interrupt access.

Navigation: Stack Monitoring → Indices / Nodes; Dev Tools → _cat/shards and _cluster/allocation/explain

Verification steps:

1. Check unassigned shard counts and recent changes.
2. If non-zero, identify whether the affected shards are PRIMARY or REPLICA.
3. Check allocation explanation, node capacity, awareness rules and recent maintenance; do not manually reassign without approval.

Expected outcome: Unassigned shards = 0 during normal conditions (exceptions documented). Stable cluster-manager/master election and no repeat disruptions.

If not OK: Use IR01; route a red cluster or critical primary failure to incident management and Elastic support.

Owner: Cloud Operations | Evidence: Shard list, allocation explanation and affected index names

D04 JVM heap, CPU and search responsiveness

Why it matters: JVM heap is the memory used by Elasticsearch processes; sustained pressure may slow searches and trigger garbage collection.

Navigation: Stack Monitoring → Elasticsearch → Nodes; Cloud performance metrics

Verification steps:

1. Inspect JVM heap utilisation and trends, CPU usage and garbage collection rate for each node.
2. Compare the same period with normal daily peaks.
3. Review slow queries, thread-pool rejections and node restarts if pressure is sustained.

Expected outcome: No persistent high-pressure trend or out-of-memory evidence; user search and ingestion response remain within agreed SLOs.

If not OK: Follow IR03; investigate expensive queries, indexing patterns and capacity. Do not alter heap settings directly on Elastic Cloud Hosted.

Owner: Cloud Operations | Evidence: Max/average heap trend and CPU charts

D05 Disk space and data-tier capacity

Why it matters: Storage fills as logs arrive; high use can prevent new data from being written.

Navigation: Stack Monitoring → Nodes / Indices; Cloud console → Elasticsearch topology / autoscaling

Verification steps:

1. Review used/free storage per hot and warm tier, not only overall cluster average.

2. Identify largest indices/data streams and unexpected daily growth.
3. Check whether Elastic autoscaling is available/enabled and whether its maximum limit leaves enough capacity.

Expected outcome: Comfortable headroom below configured Elasticsearch disk watermarks, with no write blocks or unexplained growth.

If not OK: At internal warning threshold, forecast and raise capacity action; at imminent/high watermark risk, follow IR02 and escalate. Do not delete indices ad hoc.

Owner: Cloud Operations | Evidence: Per-tier headroom, largest indices and forecast

D06 Ingestion rate and data freshness

Why it matters: Incoming records can stop even when Elasticsearch is healthy.

Navigation: Stack Monitoring → Indexing; Discover → critical service data views

Verification steps:

1. Compare index rate with normal baseline for the hour/day and watch for sudden drops or spikes.
2. Check latest event ingestion time and @timestamp for each critical service.
3. Validate source count against the approved monitoring inventory and alert thresholds.
4. If data is missing, determine whether it is a source outage, agent fault, pipeline issue, timestamp issue or permissions problem.

Expected outcome: Critical data streams receive expected records within defined freshness SLA; no unexplained gaps or mapping errors.

If not OK: Follow IR04/IR06 and involve the source-owning team. Capture first/last successful timestamps.

Owner: Observability Operations / Application owners | Evidence: Missing data list, counts, timestamps, source owner

D07 Snapshot and recovery readiness

Why it matters: Snapshots preserve recoverable copies of eligible Elasticsearch data and metadata, subject to policy/compatibility.

Navigation: Kibana → Stack Management → Snapshot and Restore / Cloud console snapshot area

Verification steps:

1. Review latest successful snapshot and last failed operation.
2. Inspect SLM policy history, repository health and expected cadence/retention.
3. Confirm any recent deployment change has a recoverable snapshot checkpoint.

Expected outcome: Recent successful snapshot within approved RPO and no unresolved backup failures.

If not OK: Open incident/change task for stale/failed backup; investigate repository/policy; do not delete managed snapshots. Follow section 10.

Owner: Cloud Operations | Evidence: Latest completed snapshot time, failed snapshot count

D08 ILM, retention and data tier movement

Why it matters: Lifecycle rules keep old data from filling fast storage and remove it when policy allows.

Navigation: Kibana → Stack Management → Index Lifecycle Policies / Data Streams; Dev Tools → _ilm/explain

Verification steps:

1. Review any ILM indices reporting ERROR and identify stuck phase/action.
2. Check expected rollover/age/size and hot/warm placement of representative critical streams.
3. Verify aged data is not held beyond the approved retention or prematurely deleted.

Expected outcome: Indices/data streams progress under their assigned lifecycle mechanism; no persistent policy errors.

If not OK: Follow IR02 or IR07. Verify whether a stream uses ILM or a different data lifecycle mechanism before changing policy.

Owner: Cloud Operations / Data governance | Evidence: Error phase/step, policy name, retention impact

D09 Fleet and Elastic Agent status

Why it matters: Fleet shows the collection software connected to hosts; unhealthy/offline collectors can lead to silent monitoring gaps.

Navigation: Kibana → Fleet → Agents

Verification steps:

1. Filter by Unhealthy, Offline, Inactive and Updating; compare with yesterday and the approved inventory.
2. Check last check-in, operating system, agent version and assigned policy for affected business-critical hosts.
3. Open agent detail for underlying component/error messages; confirm decommissioned hosts are excluded.

Expected outcome: Business-critical agents Healthy and checking in; exceptions have an owner and ticket.

If not OK: Use IR05. Do not bulk-unenroll or force-upgrade agents without change control.

Owner: Observability Operations / Host owners | Evidence: Agent status counts and exception list

D10 Beats and Logstash processing health

Why it matters: Beats are log/metric collectors; Logstash may transform and route data. Failures can disrupt only certain sources.

Navigation: Stack Monitoring → Beats / Logstash (if configured); pipeline and host logs

Verification steps:

1. Check monitored Filebeat/Metricbeat instances, restarts, output errors and newest event delivery.
2. Check Logstash pipeline throughput, worker errors, back-pressure and queue size where Logstash is actually in use.
3. Cross-check representative source events in Discover and compare against baseline volumes.

Expected outcome: Configured collectors/pipelines run normally; no unresolved output failures, growing queues or persistent mapping errors.

If not OK: Follow IR06. If Logstash/Beats is not deployed for a service, record N/A and document its ingestion route.

Owner: Observability Operations / Platform and host owners | Evidence: Pipeline rates, agent/Beat status and error examples

D11 APM, synthetics and application availability

Why it matters: APM tracks application transactions, while synthetics tests whether an endpoint works from a monitoring location.

Navigation: Kibana → Observability → APM / Services; Synthetics → Monitors

Verification steps:

1. Confirm critical applications appear within the correct time range and environment.
2. Inspect latency, throughput and error-rate changes; identify services with missing traces.
3. Review failed, disabled or stale synthetic checks, their testing locations and response errors.
4. Escalate application failures to service owners; separate application outage from observability-data collection failure.

Expected outcome: Expected applications reporting; latency/error trends within SLO; critical synthetics enabled and passing.

If not OK: Follow IR08; validate whether the service is truly down or only telemetry is missing.

Owner: Application owners / Observability Operations | Evidence: Service inventory, latency/error chart, monitor result

D12 Alert rule and notification health

Why it matters: A monitoring platform is less useful if its warnings never reach responders.

Navigation: Kibana → Stack Management → Rules / Connectors / Alerts; incident tool

Verification steps:

1. Review active critical alerts, rule execution errors and suppressed or disabled rules.
2. Check connector delivery failures and routing to approved support channels.
3. Confirm alerts have acknowledged owners and tickets; identify repeated false positives.
4. Review silences/maintenance windows for unexpected or expired exclusions.

Expected outcome: Critical rules execute, deliver to correct responders and have an explicit owner and action.

If not OK: Follow IR09; use alternate incident communications if notifications fail.

Owner: Observability Operations / Service desk | Evidence: Failed rules/connector list and actions taken

7. Preventive maintenance by frequency

7.1 Weekly: detailed review

Control	Actions and what to record	Passing result
W01 Agent/Beat hygiene	Compare offline/unhealthy/inactive agent counts by team, review outdated versions and policy drift; identify retired assets	All material exceptions assigned
W02 Pipeline quality	Review Logstash event rates, dropped/failed records, ingest processor failures, DLQ/PQ if configured, and last seven days of errors	No unowned growing failure trend
W03 Inventory and metadata	Sample important records for host.name, service.name, environment, team/owner tags and naming conventions	Required tags complete and consistent
W04 Source data freshness	Check each tier-1 source was seen within its SLA; distinguish expected quiet periods from gaps	No unexplained critical gaps
W05 ILM/storage	Review rollover failures, stuck policies, large shards, unallocated shards and hot/warm efficiency	No unowned policy/storage risk
W06 Alert effectiveness	Review false positives, missed incidents, disabled rules, thresholds and contact accuracy	Critical controls effective
W07 Change review	Correlate service changes with errors, version skew and new telemetry gaps	Regressions tracked

7.2 Monthly: service assurance

1. Review storage trend, forecast weeks-to-capacity and planned scaling ceiling; record business growth assumptions.

2. Compare actual ingest/document volume with licence/subscription and expenditure commitments where applicable.
3. Review new Elastic security/patch releases, end-of-support risk, upgrade prerequisites and external ingest component compatibility.
4. Audit users, high privilege roles, SSO mappings and all active Elasticsearch/Cloud API keys; verify expiration, least privilege and service ownership.
5. Validate snapshot success, RPO evidence and the recovery test schedule; inspect any unresolved backup alerts.
6. Review APM/trace service coverage, failed and inactive synthetics, and monitoring owner assignments.
7. Review the source inventory and CMDB alignment, data retention, sensitivity classification and decommission requests.
8. Publish monthly service report with trends, incidents, corrective actions, risk and sign-off.

7.3 Quarterly: governance and resilience

1. Validate production architecture and availability-zone resilience; confirm master/quorum, replicas and application dependency assumptions.
2. Perform a controlled snapshot restoration exercise in an isolated, approved target if policy permits; measure actual recovery times and verify restored application data.
3. Reassess alert severity/thresholds and support/incident escalation routes, including a simulated critical alert.
4. Review upgrade roadmap and vendor support windows; document tested compatibility of Logstash/Beats/agents/APM SDKs.
5. Run privilege/secret lifecycle, retention, data quality and monitoring coverage assurance reviews.
6. Retire stale agent records, policies, data streams and dashboards only under approved change with retention safeguards.
7. Review and republish this runbook after approval, material service change or post-incident learning.

8. Incident and troubleshooting playbooks

The following playbooks are designed for deep investigation. Start with read-only checks. Assign a responsible resolver, preserve the incident timeline and obtain an approved change before any destructive or system-modifying step.

IR01 Cluster yellow/red or unassigned shards

Trigger / business effect: Reduced fault tolerance (yellow) or inaccessible primary data (red).

Investigation:

1. Confirm exact time, affected index/shard, cluster status and number of unassigned primary versus replica shards.
2. Review Cloud instance events and node loss; use _cat/shards and POST /_cluster/allocation/explain with an explicit shard body if needed.
3. Check watermarks, replica count, zone awareness and node availability.

Controlled corrective actions:

1. Raise an incident based on business impact; engage Elastic support for provider/zone issues.
2. Recover lost node/capacity using approved Cloud Console procedure; request additional resources if allocation is blocked by capacity.
3. Only apply allocation configuration fixes after cause is understood and change approval obtained.

Recovery verification: Cluster returns to agreed condition; primaries active, replicas assigned, ingest and search verified.

Escalate when: Any red cluster impacting critical services, missing primary shard, quorum issue or unexplained allocation failure.

IR02 Disk pressure, write block or stuck lifecycle

Trigger / business effect: New events may stop indexing; retention or historical queries may be affected.

Investigation:

1. Inspect free storage by node/data tier, ILM/DSL policies and largest indices; identify recent ingest growth.
2. Determine whether low/high/flood-stage disk watermark is reached; check autoscaling settings and ceilings.
3. Check read-only index blocks and ILM explain error steps.

Controlled corrective actions:

1. Raise scaling request and coordinate Cloud capacity change; correct accidental ingestion spikes at source if approved.
2. Review retention with data owner; adjust lifecycle policy only after compliance/data-retention approval.
3. Remove write restrictions only via a validated, approved procedure after space is recovered; never manually delete system indices or snapshots.

Recovery verification: Disk headroom sustainable; indexing resumes; lifecycle phase progressing and source counts recovered.

Escalate when: Capacity cannot be added promptly, a flood-stage block is active, or a critical source is missing data.

IR03 High heap/CPU, slow dashboards, search delays

Trigger / business effect: Searches or ingestion slow down and JVM/CPU may be saturated.

Investigation:

1. Compare CPU, heap, garbage collection, search/index queue, rejections and node stats with baseline.
2. Identify timing of dashboard queries, bulk indexing and data-tier movement; review recent changes.
3. Check largest shards and index count; if authorised review search profiling/slow logs as available.

Controlled corrective actions:

1. Reduce or reschedule known noisy workload with owner approval.
2. Tune queries/visualisations and index/shard policies via change control.
3. Request vertical/horizontal scaling in Elastic Cloud Hosted if baseline capacity inadequate.

Recovery verification: P95 query and ingestion latencies return to agreed SLO; stable garbage collection and no rejections.

Escalate when: Out-of-memory/restarts, production-wide latency or inability to recover with approved mitigations.

IR04 Missing logs, missing metrics or delayed timestamps

Trigger / business effect: Security/operational visibility is incomplete even if the Elastic cluster is green.

Investigation:

1. Confirm Kibana data view and time picker; search for source-specific host/service dataset and compare @timestamp to event.ingested when available.
2. Verify last known event, expected event frequency, agent/Beat/Logstash health and connectivity.

3. Inspect pipeline parse/mapping errors, rejected writes, wrong environment tag, clock skew and wrong index target.

Controlled corrective actions:

1. Open source-owner incident and restore collector/pipeline connectivity under service change process.
2. Repair field mapping/parse errors in tested pipeline; replay buffered data from approved queue when possible.
3. Validate event counts and time coverage; explicitly record unrecoverable gaps.

Recovery verification: Latest events appear at expected rate, correct environment and tags; backlog drained with no duplicate spike.

Escalate when: Business-critical logs disappear beyond freshness SLA or security audit coverage is compromised.

IR05 Fleet agent unhealthy, offline, inactive or out-of-date

Trigger / business effect: Host telemetry and configuration management may be impaired.

Investigation:

1. Filter affected agents by status; review last check-in, policy, component errors, version and related host status.
2. Confirm Windows service/Linux systemd service status, TLS trust, proxy/firewall, DNS and Fleet Server endpoint reachability.
3. Check whether agent is deliberately retired or host scheduled down; review recent policy changes.

Controlled corrective actions:

1. If authorised, restart an affected agent service after collecting logs/diagnostics.
2. Correct enrollment credentials/TLS/policy permissions with owner; re-enroll only when needed and approved.
3. For version lag use a staged upgrade; account for RPM/DEB agents that require package-manager upgrades rather than Fleet UI upgrades.

Recovery verification: Agent reports Healthy, last check-in recent, policy applied and telemetry present in Discover.

Escalate when: Fleet-wide outage, critical host telemetry missing or repeated upgrade failures.

IR06 Filebeat/Metricbeat/Logstash errors or pipeline blockage

Trigger / business effect: Telemetry is delayed, dropped or routed to an incorrect destination.

Investigation:

1. Determine collection path (standalone Beat, Agent-managed integration or Logstash); do not assume all are deployed.
2. Check Beat service/log output, connection errors and config validation; review Logstash pipeline stats if present.
3. Check growing persistent queue / dead-letter queue, mapping rejects, worker exhaustion and upstream source availability.

Controlled corrective actions:

1. Apply tested output connectivity, credentials, parsing or pipeline fixes under change approval.
2. Use approved controlled retry/replay for buffered events; DLQ handling must avoid duplication and comply with retention.
3. Confirm processing rate, error count and stream landing index; restart only the failing component if warranted.

Recovery verification: Ingest pipeline steady, error/dropped event rates within baseline, queues draining and source events searchable.

Escalate when: Critical log loss, DLQ expansion with no recovery path, or sustained indexing rejection.

IR07 Data quality, metadata, retention or ILM defect

Trigger / business effect: Dashboards/alert grouping wrong, expected fields absent or data moves/deletes unexpectedly.

Investigation:

1. Sample raw events and verify @timestamp, host.name, service.name, environment, business area and expected ECS field types.
2. Inspect mapping conflict, ingest errors, agent policy version, processor changes and ILM/data lifecycle allocation.
3. Identify whether issue affects only dashboards or indexed source records; establish first bad event.

Controlled corrective actions:

1. Raise source schema/data-contract issue with owning team.
2. Test pipeline and templates against representative records in non-production before deployment.
3. Obtain compliance approval for changing retention and consider backfill/reindex only after risk assessment.

Recovery verification: Mandatory fields populated, saved searches and alerts work, and lifecycle conforms to approved data policy.

Escalate when: Regulatory retention risk, large field-loss proportion or critical dashboards misreport.

IR08 APM application not reporting or Synthetics failing

Trigger / business effect: User journeys become unobservable or actual application availability is affected.

Investigation:

1. Check correct time range and environment; APM Services lists services with recent transactions/spans, not log-only services.
2. For APM confirm SDK/agent configuration, sampling, service.name, authentication, tracing endpoint and upstream proxies.
3. For Synthetics inspect monitor enabled state, run location, HTTP/TLS/DNS result, test credentials and recent application deploy.

Controlled corrective actions:

1. If service is down, escalate to application incident owner; run independent health check.
2. Correct instrumentation/network/secret expiry with owner and change approval.
3. Repair monitor definition and re-test from approved public or private location as appropriate.

Recovery verification: Expected service traces and monitor executions return; synthetic runs pass and application owner confirms recovery.

Escalate when: Production endpoint fails from multiple credible locations, critical service has prolonged missing tracing or data cannot be trusted.

IR09 Alerting and notification failure

Trigger / business effect: Incidents may be missed even though the platform still collects data.

Investigation:

1. Inspect rule execution status, failed runs, disabled/snoozed rules and connector delivery errors.

2. Compare alerts against actual source events and recent rule configuration changes.
3. Validate incident integration or email recipient list with support owner; avoid exposing confidential payloads in tests.

Controlled corrective actions:

1. Activate approved backup paging/communication channel.
2. Repair failing action connector credentials/routing or poorly scoped rule through approved change.
3. Run a non-disruptive test alert and confirm ticket/notification receipt.

Recovery verification: Critical rules execute successfully and correct on-call recipient confirms delivery.

Escalate when: Notification outage affects all high-severity rules or incident missed its response SLA.

IR10 Unauthorised access, failed SSO or API-key incident

Trigger / business effect: Operational users may lose access or credentials may be exposed.

Investigation:

1. Determine whether fault affects Elastic Cloud organisation login, Kibana SSO or Elasticsearch API calls.
2. Review IdP availability, group/role mapping, recent changes and authentication logs in approved systems.
3. Check API key owner, privilege scope and expiration without copying secret material.

Controlled corrective actions:

1. Follow security incident process for compromised credentials; revoke/rotate affected keys and invalidate sessions where authorised.
2. Correct mappings and least-privilege role assignment via IAM approval.
3. For service account key rotation, coordinate cutover across dependent integrations and verify data still arrives.

Recovery verification: Authorised users regain intended access; unauthorised access blocked and audit evidence preserved.

Escalate when: Potential credential leak, privilege escalation or wider SSO outage.

IR11 Snapshot failure or urgent recovery request

Trigger / business effect: Data protection assurance breached or restore is needed after corruption/deletion.

Investigation:

1. Check last good snapshot, policy state, repository and backup scope, and affected data streams/indices.
2. Confirm recovery objectives, target deployment, version/index compatibility and business authority to restore.
3. Identify whether restore may overwrite existing indices/saved objects or disrupt users.

Controlled corrective actions:

1. Log incident/change and obtain data-owner approval plus recovery plan.
2. Prefer isolated restore/validation before production cutover where feasible; follow section 10.
3. After recovery, compare document counts, newest/oldest data, dashboards, security configuration and write pipelines.

Recovery verification: Recovered data confirmed by application owner; index health, search, ingestion and approved RTO/RPO evidence recorded.

Escalate when: No valid recovery point, suspected snapshot corruption or recovery may violate retention/integrity requirements.

IR12 Unexpected host/application disappearance or retirement

Trigger / business effect: An entity stops reporting, but may be decommissioned rather than faulty.

Investigation:

1. Compare latest signal with CMDB, change tickets, owner and lifecycle state.
2. Check tags/service owners and verify whether the system moved, was renamed, or is permanently retired.
3. Identify Fleet policy, monitors, alerts, dashboards, data stream retention and sensitive data dependencies.

Controlled corrective actions:

1. If not approved retirement, follow IR04/IR05 and raise data gap issue.
2. If retirement authorised, disable collection and related alerts through change; unenroll agents and revoke enrollment/API keys as appropriate.
3. Preserve or delete retained history strictly per approved retention and legal hold controls.

Recovery verification: Inventory, Fleet, alert rules and data retention accurately reflect final service state.

Escalate when: System disappears without owner confirmation or regulatory/audit data is at risk.

9. Planned change: upgrades, onboarding and decommissioning

9.1 Change classification and prerequisites

Every material upgrade or integration change requires a documented request, service impact analysis, owner approval, change window and roll-forward/restore options. Capture current versions, environment, endpoint ownership, dependencies, snapshots and test results. Avoid doing an urgent upgrade solely because a later version number exists.

9.2 Elastic Cloud Hosted deployment upgrades (CH01)

This deployment is Elastic Cloud HOSTED, so the approved upgrade path is through the Elastic Cloud Console; ScottishPower does not manually patch Elasticsearch node operating systems. Cloud-managed Elasticsearch, Kibana and associated hosted components are upgraded together according to the selected supported path.

1. Plan: review Elastic release notes, upgrade compatibility and deprecations; identify required intermediate versions and non-compatible plugins/integrations.
2. Assess: list APM SDKs, Elastic Agents, Filebeat/Metricbeat/Auditbeat, Logstash, API clients, SSO integration, ingest pipelines, Kibana dashboards, and alert connectors that require testing.
3. Baseline: record cluster green status, nodes, shard balance, heap/CPU, disk, data freshness, last successful snapshot, APM/Synthetics results and critical dashboards.
4. Preflight: use Kibana Upgrade Assistant/deprecation checks if applicable; resolve blocking issues and verify a recent recoverable snapshot.
5. Test in non-production: run representative ingestion, index templates, Logstash routing, agent policies, SSO, alerts, APM and restore verification where possible.
6. Raise/approve a change with rollback strategy focused on restoration/redeployment; Elasticsearch downgrades are not a standard in-place rollback.
7. Execute: Elastic Cloud Console → deployment → Deployment version → Upgrade; choose validated release and confirm during change window.
8. Monitor Elastic Cloud plan events, cluster state and Kibana access; investigate any failed plan before changing more components.
9. Validate functional test matrix: sources, pipelines, APM, synthetics, Kibana saved objects, security, snapshot and notifications.

10. Then upgrade externally managed ingestion components in a tested compatible order; Fleet Server/agent version compatibility needs review.
11. Close change only after sign-off, refreshed CMDB/version inventory and documented defects.

UPGRADE CAUTION Elastic documents an automatic pre-upgrade snapshot for Elastic Cloud Hosted; still verify last successful backups and recoverability. A failed upgrade or breaking change cannot generally be solved by simply selecting an older Elasticsearch version. Plan compatible restore to another deployment if necessary.

9.3 Elastic Agent upgrades — Windows and Linux (CH02)

1. Inventory agents by OS, version, installation method, Fleet-managed versus standalone, source criticality and network path to the artifact repository.
2. Choose a version compatible with Fleet Server and the hosted Elastic Stack. Confirm package support and agent policy minimum version.
3. Test one Windows non-production host and one Linux host representative of the fleet; capture before/after service status and ingested events.
4. Windows installed by supported MSI/ZIP and supported Linux tar.gz Fleet-managed installs can usually upgrade via Fleet. Linux DEB/RPM and Docker/Kubernetes installations require package/image-based upgrades as documented by Elastic.
5. Deploy in waves with clear stop criteria; do not mass-upgrade every production host at once.
6. Verify Healthy status, policy application, service process, latest logs/metrics and absence of duplicate data.
7. On failure collect diagnostics and follow documented agent-specific rollback or package remediation; notify host owner.

9.4 Beats / Logstash upgrades (CH03)

1. Verify that standalone Filebeat, Metricbeat, Auditbeat or Logstash is actually installed and record exact package, host and service owner.
2. Check Stack compatibility, plugin version and configuration deprecations; read release notes.
3. Back up configuration securely and validate pipeline syntax/configs in non-production.
4. Confirm buffers/PQ/DLQ and potential data loss exposure; obtain an ingestion pause/backfill plan if necessary.
5. Upgrade during approved window using OS package/change mechanism, not the Elastic Cloud deployment upgrade control.
6. Check collector service health, Logstash pipelines, input/output events, mapping, and last observed log after rollout.
7. Revert the component/package using tested host change procedure if permitted; restore previous configuration and confirm no telemetry loss.

9.5 New monitoring source onboarding (CH04)

1. Receive approved request with business owner, environment, host/service IDs, expected log/metric/APM sources, criticality, data classification, retention and desired alerts.
2. Select approved route: Fleet-managed Elastic Agent preferred where suitable; otherwise supported Beats, APM SDK, Logstash or API.
3. Define integration policy and secure output connectivity (TLS, proxy, firewall, credential scopes); do not embed enrollment tokens in documentation.
4. Apply naming/tags: business_unit, team, service.name, host.name, environment and dataset; align with Elastic Common Schema where practicable.

5. Test expected event sample, mapping, timestamps, data-stream naming, volume, retention and access controls.
6. Create service/dashboard/alert monitoring and define data freshness SLA and escalation ownership.
7. Obtain application owner sign-off; update source register, support inventory, diagrams and as-built configuration.

9.6 Decommissioning monitored sources (CH05)

1. Confirm decommission approval, legal hold and data-retention obligations.
2. Record last successful events, dependent rules, dashboards, Synthetics monitors, agent identities, policies and service owners.
3. Disable source collection or uninstall with host owner, then unenroll Fleet Agent or remove Beat/Logstash inputs as appropriate.
4. Remove or update obsolete alerts and dashboard dependencies; revoke no-longer-required credentials.
5. Retain historical data until lifecycle permits deletion; never purge data solely because monitoring stopped.
6. Update inventory, CMDB, ownership and closure evidence; independently confirm no unintended active systems were affected.

10. Backup, restoration and disaster recovery

10.1 What Elastic Cloud Hosted provides

Elastic Cloud Hosted automatically registers a managed snapshot repository, typically named found-snapshots, and a corresponding cloud-snapshot-policy. Elastic documentation describes a default snapshot interval of 30 minutes, subject to configuration. Verify the live schedule, last success, retention, available restore points and subscription capabilities. Snapshots are not a replacement for business-level recovery tests.

10.2 Snapshot daily/weekly assurance (BK01)

1. Read the snapshot list and SLM execution history. Confirm most recent successful snapshot is within the documented RPO.
2. Check repository status, failed attempts, deleted snapshots, unexpected retention changes and recent volume trends.
3. Confirm the snapshot includes the target indices/system data required for the intended recovery; closed indices are not backed up by ordinary snapshots in Elastic Cloud Hosted.
4. Document snapshot timestamp, repository, policy, failure count and ticket for any gap.

10.3 Controlled restore runbook (BK02)

1. Raise approved disaster recovery/restore request: affected service, exact index/data stream, incident and data owner.
2. Identify last known good restore point and acceptable data loss based on agreed RPO.
3. Validate target Elasticsearch version compatibility, repository access, security controls, index naming conflicts and dependencies.
4. Select separate recovery deployment where feasible; plan access restriction and cost/capacity.
5. Ask Elastic support to assist when managed features or repository access require it.
6. Restore only intended indices/data streams/system feature states under approved documented plan; avoid blindly restoring the entire cluster state to active production.
7. Test reads, document counts, timestamp distribution, sample business records, Kibana objects, role mapping, APM ingestion and alerts.

8. Switch dependent ingest/search workflows if recovery requires a cutover; capture actual RTO and RPO, data gap and business sign-off.

10.4 Disaster recovery decision table

Scenario	First response	Recovery strategy
Temporary AZ/node issue	Confirm Cloud activity and shard/replica status	Provider recovery/scale; validate replica availability
Accidental index deletion	Freeze further changes, identify index and latest recovery snapshot	Restore selected data to isolated destination first
Corrupted mapping or bad ingest change	Stop offending pipeline change, determine first bad write	Rollback policy/pipeline and replay or restore affected range if approved
Major platform/version incompatibility	Open incident and support case	Build compatible target and restore from valid snapshot if required
All ingestion stopped but indices healthy	Trace source-to-index route	Restore collectors/pipelines; replay buffers if available

CRITICAL LIMITATION Do not remove or delete the managed found-snapshots repository/snapshots casually: Elastic notes that deleted managed snapshots are not recoverable from an independent storage-level version. Additional separate repositories may be needed to satisfy stricter enterprise resilience requirements.

11. Thresholds, alerts, evidence and reporting

11.1 Monitoring threshold catalogue — initial policy only

Signal	Early attention / amber	Critical decision	Notes
Cluster status	Yellow or unexpected unassigned replica shards	Red with unassigned primary / business impact	Investigate even temporary yellow; assess actual impact
JVM heap	Sustained >75% and rising from baseline	Sustained >85% with GC/rejections/latency	Avoid judging from momentary spikes
Disk used / tier	≥70% internal capacity review; rising forecast	≥80% internal high-risk; immediate review of configured watermarks	Elasticsearch common defaults low/high/flood 85/90/95%; headroom rules apply
CPU	Sustained high versus historical baseline	High with blocked queues/rejections or SLO breach	CPU alone is insufficient
Unassigned shards	Any unplanned >0	Critical primary unavailable or sustained impact	Read routing explanation
Critical data freshness	No events within each source-specific SLA	Critical/security source exceeds agreed outage threshold	Exclude approved idle sources
Snapshot RPO	Last success approaches RPO	Last success exceeds RPO / no restorable snapshot	Live schedule must be verified
Fleet agents	Unhealthy/offline on a	Widespread gap or lost	Inactive may be

Signal	Early attention / amber	Critical decision	Notes
	monitored critical host	tier-1 telemetry	intentionally dormant
Synthetics	Isolated failure pending confirm	Repeated failures from credible locations and business impact	Check monitor design/credentials
Alert connectors	Delivery error / paused rule	Critical paging unavailable	Exercise alternate escalation

These values are policy proposals inspired by the existing runbook, not vendor-wide SLOs. A watermarked Elasticsearch volume may have other limits based on absolute disk headroom, data tier and overridden settings. See official threshold documentation.

11.2 Standard alert record

Field	Required content
Identity	Rule ID, deployment, environment, monitored system/data stream
Condition	Signal, threshold, duration, evaluation interval, baseline
Severity	P1/P2/P3/P4 mapping agreed with service owner
Response	Who gets notified, connector, backup method, route to runbook step
Noise control	Deduplication, maintenance windows, recovery conditions
Test	Date, evidence link, outcome, owner
Review	Owner and last review date

11.3 Daily health record (copyable template)

Field	Example or entry
Check date / shift	[YYYY-MM-DD HH:MM UTC]
Environment / deployment	[Production / exact ID]
Reviewer	[Team and named operator]
Cloud / cluster / nodes	[Green/Amber/Red; key values]
Shards / disk / heap	[Values per affected tier]
Ingestion / gaps	[Critical datasets checked; last seen times]
Fleet / pipeline	[Agents unhealthy/offline; queues/errors]
Snapshots / ILM	[Last success; lifecycle error count]
APM / Synthetics / alerts	[Service availability; monitor failures; connector state]
Open incidents and actions	[Ticket; priority; owner; next update]
Sign-off	[Reviewer and time]

11.4 Incident checklist and post-incident review

1. Declare priority and incident owner using ScottishPower process, including timestamp and scope.
2. Establish platform versus source versus application fault; capture first and last affected times.
3. Record business impact, affected services/hosts/data streams and source coverage loss.
4. Preserve evidence and notify internal stakeholders and Elastic support as needed.
5. Document actions and approvals chronologically; capture change/command audit trail.
6. Confirm recoveries with end-to-end checks, not only green cluster status.
7. After resolution, perform RCA, identify alert/runbook gaps and assign prevention actions with due dates.

12. Completion checklist, open decisions and glossary

12.1 Runbook readiness / pre-production acceptance

Acceptance requirement	Status
Live production architecture, all zone roles and master quorum validated	[OPEN]
Service and support URL register confirmed from Cloud Console	[OPEN]
Named technical/business owners and RACI signed off	[OPEN]
RTO/RPO, backup retention and recovery tests approved	[OPEN]
P1–P4 SLA, contact tree, service desk and Elastic support entitlements approved	[OPEN]
Source inventory, criticality and freshness SLAs documented	[OPEN]
Thresholds baseline-tested and alerts have accountable owners	[OPEN]
Fleet/Beats/Logstash locations and version management method confirmed	[OPEN]
SSO, Cloud-vs-deployment access, API-key standards approved	[OPEN]
Non-production restore and upgrade rehearsal completed	[OPEN]
Review, approval, publication and next review date recorded	[OPEN]

12.2 Important open design questions

- Is Stack Monitoring data written to the production deployment or a separate monitoring deployment? If production fails, can operators still see observability data?
- Does the current design truly have three-zone high availability for master-eligible nodes and Kibana/Integrations Server? Is ML highly available if required?
- Which telemetry paths actually use Logstash, standalone Beats, Fleet-managed Elastic Agent and APM? Identify the specific hosts and owners.
- Are index lifecycle policies managed using classic ILM or data stream lifecycle, and are hot/warm tiers aligned with the actual billing/capacity strategy?
- Which Synthetics monitors are public versus Private Locations, and which are business critical?

- What is the contracted Elastic Cloud support entitlement, response target and case-escalation route?
- What retention and classification apply to audit/security events, personally identifiable data and application logs?
- What is the approved deployment upgrade cadence and supported compatibility matrix for all dependent agents?

12.3 Concise glossary

Acronym	Meaning
APM	Application Performance Monitoring
AWS / AZ	Amazon Web Services / Availability Zone
CMDB	Configuration Management Database
DLQ / PQ	Dead-Letter Queue / Persistent Queue
ECS	Elastic Common Schema
ECH	Elastic Cloud Hosted
ILM / SLM	Index Lifecycle Management / Snapshot Lifecycle Management
JVM / GC	Java Virtual Machine / Garbage Collection
KPI / SLO	Key Performance Indicator / Service Level Objective
RACI	Responsible, Accountable, Consulted, Informed
RCA	Root Cause Analysis
RPO / RTO	Recovery Point Objective (data loss tolerance) / Recovery Time Objective (time to recover)
SSO / SAML	Single Sign-On / Security Assertion Markup Language

Appendix A. Read-only diagnostic command catalogue

Run in Kibana → Dev Tools → Console against the approved deployment. These examples are for skilled operators with the required least-privilege role. They do not modify the cluster. Large wildcard queries and verbose API responses can still consume resources; limit output and inspect scope.

Purpose	Example (Kibana Dev Tools)	How to interpret
Cluster overall	GET /_cluster/health?pretty	status, number_of_nodes, unassigned_shards
Node usage	GET /_cat/nodes?v&h=name,ip,node.role,cpu,heap.percent,ram.percent,disk.used_percent	Check per-node pressure; fields vary by version
Data tier allocation	GET /_cat/allocation?v	Look for disk and shard skew
Shard assignment	GET /_cat/shards?v&s=state	Locate UNASSIGNED shards / primary p or replica r
Allocation explanation	POST /_cluster/allocation/explain	Diagnostic request; POST is read-only but may require an explicit shard body
Indices by space	GET /_cat/indices?v&s=store.size:desc	Identify high-space data; system

Purpose	Example (Kibana Dev Tools)	How to interpret
		indices may be hidden
Index stats	GET /_nodes/stats/indices/indexing,search	Look for rate, times and errors
Node stats	GET /_nodes/stats/jvm,fs,os,process	Heap, free disk, OS stats
Lifecycle issues	GET /logs-*/_ilm/explain?human	Applicable only where ILM is actually used
Lifecycle policies	GET /_ilm/policy	Policies and configured actions
Snapshots	GET /_slm/stats	Snapshot lifecycle metrics if authorised/configured
Repository list	GET /_snapshot/_all	Configured repositories, not backup success itself
Security identity	GET /_security/_authenticate	Shows current authenticated identity and roles
Ingest pipeline list	GET /_ingest/pipeline	Review Elasticsearch ingest processors

A.1 Explicit allocation explanation example

Diagnostic POST examples are permitted only when operator-approved. This endpoint requests an explanation; it does not reroute shards. If identifying an unassigned shard, use known index, shard number and primary flag. An empty request may explain one unassigned shard, depending on API behaviour.

```
POST /_cluster/allocation/explain
{
 "index": "<index-name>",
 "shard": 0,
 "primary": false
}
```

A.2 Host-side checks (only on systems ScottishPower controls)

Elastic Cloud Hosted nodes are managed by Elastic. Do NOT try to SSH into Elastic-managed nodes; the following checks apply only to ScottishPower-operated agents, Beats or Logstash hosts:

```
Linux (with authorised access):
 systemctl status elastic-agent
 elastic-agent status
 systemctl status filebeat
 systemctl status logstash

Windows PowerShell (with authorised access):
 Get-Service elastic-agent
 Get-Service filebeat
 Get-Service metricbeat
```

Service names may differ by installation. Use approved secured logs and installation-specific configuration. For standalone Beats, use version-appropriate config validation (for example, filebeat test config) in an authorised environment. Logstash typically exposes local node/pipeline stats on its configured monitoring API, often port 9600; never expose that endpoint publicly or assume it is enabled.

A.3 How to interpret common API results

Finding	Interpretation	First action
status: green	Primary and replica shard assignments healthy at the instant sampled	Still check source freshness
status: yellow	Some replicas unassigned; resilience reduced	Inspect allocation
status: red	At least one primary shard unassigned; some data may be unavailable	Incident/escalate
unassigned_shards > 0	Allocation issue may be temporary or sustained	Identify index, primary/replica and cause
High heap with low CPU	May be normal caching or memory pressure	Review duration, GC, rejections and latency
Last @timestamp old	Could be truly missing events, quiet source or timestamp skew	Check event.ingested/agent and expected frequency

Appendix B. Official references and best-practice basis

These references support the operating principles; check version-specific applicability and licence/subscription features before applying any procedure.

01. Elastic Cloud Hosted overview: https://www.elastic.co/docs/deploy-manage/deploy/elastic-cloud/cloud-hosted

02. Manage hosted deployments: https://www.elastic.co/docs/deploy-manage/deploy/elastic-cloud/manage-deployments

03. Upgrade on Elastic Cloud Hosted: https://www.elastic.co/docs/deploy-manage/upgrade/deployment-or-cluster/upgrade-on-ech

04. Stack monitoring: https://www.elastic.co/docs/deploy-manage/monitor/stack-monitoring

05. Disk-based shard allocation and watermarks: https://www.elastic.co/docs/reference/elasticsearch/configuration-reference/cluster-level-shard-allocation-routing-settings

06. Elasticsearch REST APIs: https://www.elastic.co/docs/reference/elasticsearch/rest-apis

07. ILM explain API: https://www.elastic.co/docs/reference/elasticsearch/rest-apis/explain-lifecycle

08. Elastic Cloud Hosted snapshot repositories: https://www.elastic.co/docs/deploy-manage/tools/snapshot-and-restore/elastic-cloud-hosted

09. Fleet agent status and monitoring: https://www.elastic.co/docs/reference/fleet/monitor-elastic-agent

10. Upgrade Fleet-managed Agents: https://www.elastic.co/docs/reference/fleet/upgrade-elastic-agent

11. Fleet agent policies: https://www.elastic.co/docs/reference/fleet/agent-policy

12. Logstash queues and data resiliency: https://www.elastic.co/docs/reference/logstash/queues-data-resiliency

13. Logstash dead letter queues: https://www.elastic.co/docs/reference/logstash/dead-letter-queues

14. APM service inventory: https://www.elastic.co/docs/solutions/observability/apm/services

15. Synthetic monitoring: https://www.elastic.co/docs/solutions/observability/synthetics

16. Elastic Cloud organisation access: https://www.elastic.co/docs/deploy-manage/users-roles/cloud-organization

Approval record

Role	Name / sign-off / date
Document author	[CONFIRM]
Observability operations reviewer	[CONFIRM]
Cybersecurity and IAM reviewer	[CONFIRM]
Cloud service owner	[CONFIRM]
Business/service owner	[CONFIRM]